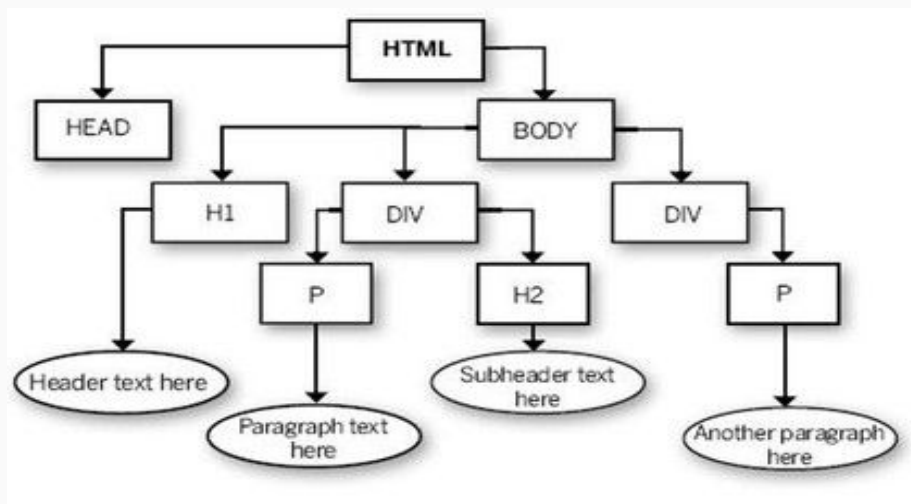


DOM и BOM

Document object model

HTML-код, который вы пишете, анализируется и преобразуется браузером впоследствии в DOM



Дочерние, соседние

Узлы элементы - теги.

Текстовые узлы - текст.

DOM

Узлы DOM.

JS может работать с элементами DOM и изменять их. соответственно и код меняется.

```
<div id="container"></div>
```

```
<script>  
    var container = document.getElementById("container");  
    container.innerHTML = "Абра-кадабра!";  
</script>
```

DOM

При генерации DOM дерева исправляются ошибки в документе, закрываются теги, ставятся токи с запятой и т.д.

DOM нужен для управления страницей - читать информацию из HTML, создавать новые элементы и изменять существующие.

```
<div id="my"></div>
```

```
document.getElementById("my").style.backgroundColor = "green";
```

Проверка ошибок JS

<http://jshint.com/>

Доступ к DOM элементам

Начинается с document

document.body - доступ к body

Узел не найден в DOM равен null

document.getElementById("idname"); - по id

document.getElementsByClassName("classname"); - по классу

работа с DOM

`document.getElementsByTagName("div");` - по тегу

`document.getElementsByName("name");` - по имени

`document.querySelector("div#main>p");` //Вернет `<p></p>` который находится внутри `<div id="main">`

`document.querySelector("div#main+p");` //Вернет `<p></p>` который идет после `<div id="main">`

Работа с DOM

```
document.querySelectorAll("p.message"); //Вернет все p у которых class="message"
```

Доступ к родительским и дочерним элементам

```
document.body.firstChild //Вернет первый дочерний элемент
```

```
document.body.lastChild //Вернет последний дочерний элемент
```

```
document.body.parentNode //Доступ к родительскому элементу (для body это html)
```

```
document.body.nextSibling //Доступ к следующему соседнему элементу - head
```

```
document.head.previousSibling //Доступ к предыдущему соседнему элементу - если  
после html есть разрыв строки, то вернет его, если нет - то вернет null
```


Работа с DOM

`document.body.firstChild` //Вернет первый дочерний html элемент

`document.body.lastElementChild` //Вернет последний дочерний html элемент

`document.body.parentNode` //Вернет родительский html элемент

`document.body.nextElementSibling` //Вернет следующий соседний html элемент

`document.head.previousElementSibling` //Вернет предыдущий соседний html элемент

Работа с DOM

Получение и перезапись содержимого тегов на странице с помощью javascript

`div.innerHTML` //Вернет строку со всеми элементами содержащимися в ноде (в данном случае в `div`)

`div.innerHTML = "<p>Hello!</p>"` //Также можно их перезаписать, что изменит контент страницы

`input.value = "blabla"` //То же самое, но для формы ввода

Работа с DOM

Доступ к тексту внутри элементов

```
<p id="my">Hi HI HI</p>
```

```
document.write(document.querySelector("#my").innerText); // Hi Hi Hi
```

в некоторых браузерах textContent

работа с DOM

Объект `style` позволяет получить доступ к каскадным таблицам стилей.

Свойства соответствуют атрибутам в каскадных таблицах стилей с небольшими отличиями в написании:

- символы "-" удаляются;
- первые буквы всех слов в названии атрибута, кроме первого, делаются прописными.

Приведем примеры преобразования атрибутов стиля в свойства объекта `style`:

`color` -> `color` `font-family` -> `fontFamily` `font-size` -> `fontSize`

`border-left-style` -> `borderLeftStyle`

работа с DOM

[createElement](#) Создает элемент.

```
document.getElementById("id").style.CSS_свойство="значение"
```

```
element.style.width="100px"
```

```
style.cssFloat вместо style.float
```

```
button.style.MozBorderRadius = '5px';
```

свойства с префиксами

```
button.style.WebkitBorderRadius = '5px';
```

Вывод на экран всех свойств объекта

```
var obj = {  
  
    Ivan: 30,  
  
    Miho: 55,  
  
    Lev: 19  
  
};  
  
for (var prop in obj) {  
  
    document.write(prop + " = " + obj[prop] + '<br/>');  
  
}
```

Browser Object Model

С помощью объектной модели браузера (BOM) Вы можете управлять поведением браузера из JavaScript.

BOM

Browser Object Model (BOM)



Объект window

Объект window является корневым объектом JavaScript. Все объекты JavaScript, а также переменные и функции определяемые пользователем хранятся в объекте window.

Посмотрите свойства объекта window

`window.open('http://vk.com');` - сайт откроется в новом окне

при обращении к объектам и переменным можно не писать window.

window

Объект Window

Для каждого открытого окна браузер создает объект Window

Объект Window включает перечисленные ниже объекты

Объект Location

Объект Location содержит информацию о текущем URL.

Объект Navigator

Объект Navigator содержит информацию о браузере пользователя

Объект History

Объект History содержит URL которые были посещены пользователем
(в данном окне браузера)

Объект Screen

Объект Screen содержит информацию о экране пользователя

Объект Frames

Объект Frames содержит информацию о всех frame и iframe

Объекты DOM

Объекты JavaScript

window

- Объект [navigator](#) содержит общую информацию о браузере и операционной системе. Особенно примечательны два свойства: `navigator.userAgent` — содержит информацию о браузере и `navigator.platform` — содержит информацию о платформе, позволяет различать Windows/Linux/Mac и т.п.
- Объект [location](#) содержит информацию о текущем URL страницы и позволяет перенаправить посетителя на новый URL. `document.write(location.href);`
- Функции `alert/confirm/prompt` — тоже входят в BOM.

window

//Обратимся к объекту navigator

```
document.write(window.navigator.appName+'<br />');
```

//Теперь обратимся к объекту navigator опустив window

```
document.write(navigator.appName);
```

//Создадим переменную a (то же самое что a=10)

```
window.a=10;
```

//Выведем значение переменной a на экран

```
document.write(a+'<br />');
```

```
document.write(window.a);
```

window методы

alert()

open()

//Откроем новое пустое окно

nw=open();

//Выведем сообщение в новое окно

nw.document.write('Этот текст был выведен с помощью JavaScript.');

window методы close() и print()

```
<script type=text/javascript>
//Откроем новое окно
nw=open();
//Выведем сообщение в новое окно
nw.document.write('Этот текст был выведен с помощью JavaScript.');
```

/* Создадим функцию cl() закрывающую окно nw
которая будет вызываться после нажатия на кнопку */

```
function cl(){
    nw.close();}
</script>
<input type='button' value='Закрыть окно' onclick='cl()' />
```

window свойства

closed Возвращает логическое значение (true или false) в зависимости от того, закрыто указанное окно или открыто.

frames Возвращает массив всех фреймов на странице (включая iframe).

document Возвращает объект Document данного окна.

history Возвращает объект History данного окна.

length Возвращает количество фреймов (включая iframe), которые находятся в данном окне.

location Возвращает объект Location данного окна.

name Устанавливает или возвращает имя данного окна.

navigator Возвращает объект Navigator данного окна.

opener Возвращает ссылку на окно, которое открыло данное.

parent Возвращает родительское окно данного окна.

screen Возвращает объект Screen данного окна.

self Возвращает текущее окно.

top Возвращает верхнее браузерное окно для данного окна.

window методы

`alert()` Вызывает окно оповещения, которое содержит текст сообщения и клавишу ОК.

`blur()` Делает окно неактивным.

`clearInterval()` Прекращает повторное выполнение кода заданного `setInterval()`.

`clearTimeout()` Отменяет запланированное методом `setTimeout()` выполнение кода.

`close()` Закрывает окно.

`confirm()` Вызывает окно подтверждения содержащее текст сообщения и клавиши ОК и Отмена.

`focus()` Делает окно активным.

`moveBy()` Смещает окно относительно его текущей позиции.

`moveTo()` Перемещает окно на указанную позицию.

`open()` Открывает новое окно браузера.

`print()` Распечатывает содержимое текущего окна.

`prompt()` Вызывает окно запроса, побуждающее посетителя ввести в него определенные данные.

`scrollBy()` Прокручивает содержимое окна на указанное количество пикселей.

`scrollTo()` Прокручивает содержимое окна до указанных координат.

`setInterval()` Вызывает функцию или выполняет код через определенные промежутки времени (указанные в миллисекундах).

`setTimeout()` Вызывает функцию или выполняет код после указанного количества миллисекунд один раз.

объект navigator

С помощью объекта navigator Вы можете определить какой браузер использует пользователь.

Так же с помощью navigator Вы можете проверить может ли пользователь принимать cookie и включена ли у него поддержка Java.

navigator свойства

userAgent

//Выведем информацию о Вашем браузере на страницу

```
document.write('Информация о Вашем браузере: '+navigator.userAgent);
```

С помощью свойства `cookieEnabled` Вы можете узнать включена ли возможность использования `cookie` в браузере пользователя. Свойство возвращает `true` если возможность использования `cookie` включена и `false` если нет.

navigator свойства

```
//Проверим может ли Ваш браузер принимать cookie  
if (navigator.cookieEnabled)  
    document.write('Ваш браузер может принимать cookie!');  
else  
    document.write('Ваш браузер не может принимать cookie.');
```

С помощью свойства appVersion Вы можете узнать версию браузера.

navigator свойства

//Узнаем версию Вашего браузера и выведем ее на экран

```
document.write(navigator.appVersion);
```

С помощью свойства **appName** Вы можете узнать название браузера, а с помощью **appCodeName** кодовое название браузера.

//Узнаем название Вашего браузера и выведем его на страницу

```
document.write(navigator.appName + '<br />');
```

//Узнаем кодовое название браузера и выведем его на страницу

```
document.write(navigator.appCodeName + '<br />');
```

navigator свойства

С помощью свойства `platform` Вы можете узнать платформу, под которую скомпилирован браузер (т.е. узнать ОС, которую использует пользователь).

//Узнаем платформу, под которую скомпилирован Ваш браузер и выведем ее на страницу
`document.write(navigator.platform);`

navigator метод

С помощью метода `javaEnabled()` Вы можете проверить включена ли поддержка Java в браузере пользователя или нет. Метод вернет `true` если поддержка включена и `false` если нет.

```
//Проверим включена ли в Вашем браузере поддержка Java  
document.write(navigator.javaEnabled());
```

Объект screen

Объект Screen содержит информацию об экране пользователя.

С помощью свойств данного объекта Вы можете узнать какое разрешение, а также какая глубина цвета установлена на экране пользователя.

screen свойства

Свойство
Описание

[availHeight](#) Возвращает высоту экрана (исключая такие элементы интерфейса как полоса прокрутки, строка состояния, панель инструментов и т.д.).

[availWidth](#) Возвращает ширину экрана (исключая такие элементы интерфейса как полоса прокрутки, строка состояния, панель инструментов и т.д.).

[colorDepth](#) Возвращает битовую глубину цветовой палитры для отображения изображений.

[height](#) Возвращает общую высоту экрана.

[width](#) Возвращает общую ширину экрана.

screen свойства

//Узнаем разрешение Вашего экрана и выведем его на на страницу
document.write('Разрешение экрана: ' + screen.width + 'X' +
screen.height + '');

//Узнаем глубину цвета Вашего экрана и выведем ее на страницу
document.write('Глубина цвета: ' + screen.colorDepth);

объект history

Объект History содержит список URL которые были посещены в данном окне браузера.

history свойства и методы

С помощью свойства **length** Вы можете узнать количество посещенных URL хранящихся в списке.

```
//Выведем количество посещенных в данном окне браузера URL  
document.write(history.length);
```

Вы можете перемещаться по списку посещенных URL с помощью метода **go(смещение)**.

Например, с помощью go(2) Вы можете переместитя на два URL вперед, а go(-3) переместит Вас на три URL назад.

```
function bc2() {  
    window.history.go(-2);  
}
```

history методы

[back\(\)](#) Загружает предыдущий URL в списке хранимых URL.

[forward\(\)](#) Загружает следующий URL в списке хранимых URL

объект Location

С помощью объекта location Вы можете узнать любую информацию о URL текущего документа.

location свойства

Свойство **href** хранит URL текущего документа целиком.

```
document.write(location.href);
```

С помощью свойства **pathname** Вы можете узнать путь к загруженному документу.

```
document.write(location.pathname);
```

Свойство **host** содержит имя домена загруженного документа.

```
document.write(location.host);
```

location свойства

[hash](#) Возвращает часть URL содержащую якорь.

[protocol](#) Возвращает часть URL содержащую название протокола.

[search](#) Возвращает часть URL содержащую передаваемые запросы.

location методы

С помощью метода **assign()** Вы можете загрузить новый документ в данное окно браузера (изменить текущий URL на желаемый).

//После нажатия на кнопку будет загружена главная страница нашего сайта

```
function newdoc (){  
    location.assign('http://www.wisdomweb.ru');  
}  
<input type='button' value='Загрузить главную страницу' onclick='newdoc()' />
```

[reload\(\)](#) Загружает документ заново.